Sanjeevani ML project: synthetic outbreak early warning and detection

Canonical project document · 4 November 2026

This Markdown file is the single source for the college ML scope, data provenance, reproduction steps, evaluation rules, application link, and remaining work. The PDF beside it is a generated reading copy. The code lives in `ml/` of github.com/Yash13606/sanjeevani.

1. The problem and the final idea

Public health teams may have limited time to examine every area when several weak signals arrive. This project asks whether a model can **rank simulated districts for investigation** using observations available at a weekly decision time. The project is a controlled simulation study for a college ML course. It does not predict an actual district's outbreak or determine whether water is safe.

The final ML idea has two binary classification tasks:

Task	Decision at Monday `t`	Eligible rows	Ground truth
M1, early warning	Will a new simulated outbreak start during `(t, t+7 days]`?	Synthetic units with no active event at `t`	Hidden event onset after `t`
M2, current detection	Is a simulated outbreak active at `t`?	All synthetic unit weeks	Hidden event start and end around `t`

A model produces a score between 0 and 1 for each synthetic unit-week. Review flags are derived from thresholds selected on validation worlds. These are **experimental scores**, not alerts to citizens, clinical diagnoses, or official surveillance statements.

2. What the data is and where it comes from

The final training and test datasets are fully synthetic. The code creates independent Tamil Nadu-style worlds containing district profiles, climate, water-system changes, simulated infections and symptoms, event truth, and observed signals. The three surveillance scenarios (baseline, improved, enhanced) change observation coverage/delay while using the same hidden truth within a world. The current configuration contains **43 synthetic units** and uses a six-year daily timeline with a one-year warm-up. These are generated units, not the official 43 districts or measured conditions of Tamil Nadu.

Path in this Git repository	Role
`ml/sanjeevani_sim/`	Generates a world and its hidden event truth and observed signals
`ml/config/tamil_nadu_districts.yaml`	Synthetic unit profiles and relationships
`ml/config/smoke.yaml`	One train, one validation, one test world for a code check
`ml/config/simulation_config.yaml`	Planned 20 train, 5 validation, 10 test world study

Path in this Git repository	Role
`ml/sample_data/`	120 synthetic rows sampled from **training** world 1001, baseline scenario, with hash/provenance; a schema preview only
`ml/sanjeevani_ml/`	Labels, as-of features, baselines, model fitting, evaluation
`ml/data_provenance.json`	Machine-readable synthetic classification and source context
`ml/runs/<run-id>/ml_datasets/`	Generated world cohort tables; created on a laptop, ignored by Git
`ml/runs/<run-id>/results/`	Metrics, held-out scores, and run manifest; generated and ignored by Git
`ml/runs/<run-id>/models/`	Saved model artifacts; generated and ignored by Git

The earlier hybrid experiment used EpiClim's historical IDSP compilation, NFHS-5 district indicators, and NASA POWER weather. Those source files are historical context and **are not used as training features or labels in the final simulation-only version**. Earlier hybrid tables, six model files, and their reported scores are intentionally absent from this Git handoff because their audit identified validity problems. A separate AstraxHealth Next.js checkout and its four-week model are also outside this repository's ML pipeline.

The generated data are governed by documented assumptions. Reproducibility does not make them representative of real outbreak prevalence, reporting behavior, climate relationships, or intervention effects. No real patient records are needed or accepted by the ML pipeline.

3. How the code turns data into a prediction

1. **Generate independent worlds.** Each seed yields a separate synthetic world with a hidden event registry and observed surveillance scenario files.
2. **Sample weekly decision times.** Score at the **end of each configured Monday**, after that day's observations are available and after warm-up. Features can use only observations available by `t`; hidden truth, future observations, and future backfill are excluded.
3. **Construct labels.** M1 looks forward seven days for a new onset only among units without an active event. M2 checks whether an event's start/end span contains `t`.
4. **Fit on training worlds.** Fit simple baselines and classification models. A handwritten NumPy logistic regression is included for the course's mathematical/implementation requirement, alongside library models for comparison.
5. **Choose on validation worlds.** Select model and operating thresholds with validation data. The test-world labels are used only for the final evaluation.
6. **Save evidence.** A run folder contains generated cohorts, predictions, numerical metrics, model artifacts, configuration and source hashes, package versions, and a provenance manifest.

A world, rather than an individual row, is the split unit. Baseline, improved, and enhanced observations of one hidden world remain in the same split. This prevents one world's common event truth from appearing on both sides of an evaluation split.

The operational onset threshold is derived from the simulator's warm-up period only. A stricter first-case onset is retained as a sensitivity label. Case observations may appear before the later operational threshold is crossed, so the analysis must state which onset definition it evaluates.

Handwritten logistic regression mathematics

For an input vector `x_i`, the scratch model forms `z_i = w dot x_i + b` and `p_i = 1 / (1 + exp(-z_i))`. With binary target `y_i`, class weight `a_i`, and regularization coefficient `lambda`, it minimizes the average weighted binary cross-entropy plus `lambda/2 * ||w||^2`. The derivative of one unweighted loss with respect to `z_i` is `p_i - y_i`. Applying the chain rule gives the batch gradient `grad_w = mean(a_i * (p_i - y_i) * x_i) + lambda * w` and `grad_b = mean(a_i * (p_i - y_i))`. Each step subtracts the learning rate times these gradients. Missing-value medians and scaling statistics are fitted on training worlds only. The same held-out worlds are used to compare this handwritten estimator with the library logistic regression.

The comparison includes a seasonal-rate baseline, a simple weather/sensor rule, handwritten logistic regression, library logistic regression, random forest, and XGBoost when its pinned package is available. Their preprocessing is trained only on training worlds. The chosen classifier is selected by validation average precision, and the selected result is evaluated once on the held-out test worlds. The baselines remain in the results as comparison points even if a classifier is selected.

4. How to reproduce on a friend's Windows laptop

Requirements: Git, Node.js 20 or newer, and Python 3.12 or 3.13. Use a clean PowerShell terminal:

```
git clone https://github.com/Yash13606/sanjeevani.git
cd sanjeevani
npm ci
npm run build
cd ml
.\setup.ps1
.\run.ps1 -Mode Test
.\run.ps1 -Mode Smoke
```

`setup.ps1` creates `ml/.venv/` and installs pinned Python requirements. `-Mode Test` checks the pipeline guards. `-Mode Smoke` creates a timestamped folder below `ml/runs/` with three independent worlds, generated datasets, and evaluation output. It demonstrates that the pipeline runs; it is not a stable estimate of performance.

After reviewing the simulator/configuration and freezing model choices, the larger experiment is:

```
.\run.ps1 -Mode Full
```

It uses 20/5/10 configured seeds and may require substantial CPU, RAM, disk, and time. Keep its complete run folder; the manifest links every result to the inputs and versions that produced it. Do not tune parameters against the test worlds after viewing their results.

For the UI demonstration, start the local read-only result viewer and React development server in separate terminals:

```
cd sanjeevani\ml
.\.venv\Scripts\python.exe service.py --verbose
```

```
cd sanjeevani
npm run dev
```

Open `http://localhost:8080` and select **Simulation Lab**. The local service serves the newest completed run at `http://127.0.0.1:8000`. Its status endpoint shows unavailable until one exists. The page reads saved synthetic test-world scores and can recalculate a selected row with the saved model and its generated as-of features. It does not perform live surveillance or create citizen alerts.

The local API contract is `GET /api/ml/status`, `GET /api/ml/summary`, `GET /api/ml/predictions?task=m1|m2&limit=1..100`, and `GET /api/ml/infer` with a saved test-world ID, scenario, unit ID, date, and task. The run outputs include `benchmark_metrics_summary.csv`, `event_metrics_summary.csv`, `test_predictions_m1.csv.gz`, `test_predictions_m2.csv.gz`, and `run_manifest.json` under `results/`. The manifest records the exact configuration and provenance. Python and UI tests should pass before handing a run to a teammate.

5. Evaluation and what a result means

For each task, scenario, and model, report:

- positive prevalence and PR-AUC (average precision), together;
- precision, recall, F1, calibration/Brier, and false alarms per district-month at the validation-selected threshold;
- event recall and lead/delay relative to the specified simulated onset or simulated formal report date, with the denominator and missed events;
- spread of performance across independent **test worlds**, rather than one pooled number alone.

M1 warning lead is measured **before simulated onset**. M2's potential lead over a formal report is measured **before simulated reporting**; it is a different clock and must never be described as lead before onset. A model score can rank areas for review within the synthetic experiment. It says nothing about real-world accuracy until linked real data and independent validation are obtained.

The full 20/5/10 study has to be run and reviewed before any final performance number is quoted. A smoke-run score is only a software check. The prior hybrid study's approximate M1 PR-AUC 0.14/0.18/0.32 and M2 0.53/0.66/0.71 are archived historical results with different labels and simulator defects; they are **not** the outcome of this pipeline.

Verified smoke run, 4 November 2026: Three independent worlds produced nine scenario cohorts (11,137 unit-weeks each), both saved models, result tables, and a hash-checked provenance manifest. The one test world yielded the following pooled-scenario numbers after selection on one validation world. They demonstrate execution only and are **not final model performance**.

Smoke task	Selected model	Eligible test rows	Positive rate	Average precision
M1	XGBoost	24,591	0.052	0.545
M2	XGBoost	33,411	0.264	0.771

In the final Git layout, 13 Python tests passed from both the repository root and the `ml` directory. The React production build and full lint command passed; lint reported nine non-blocking Fast Refresh warnings. The local API read the smoke run and recomputed a saved test-row score from the actual model to numerical precision. A browser visual check was unavailable in this session, so that UI interaction remains for the teammate to confirm on their laptop.

6. Course deliverables and remaining evidence

The signed 21CSC307P plan expects a problem and dataset statistics, a mathematical model plus a handwritten Python algorithm without an ML estimator library, algorithm comparison, a running demonstration, GitHub link, and a final report of at most 30 pages. This document gives the scope and method. The run manifest, dataset counts, model outputs, plots, and actual measurements from the full run must be inserted into the final college submission. The presentation should name the synthetic setting on every result slide.

The coding handoff is complete when the checked-in source passes tests and a small run completes on a fresh environment. The scientific study remains conditional on a frozen full run and review of its results. Real public-health use would require lawful linked surveillance and water observations, reporting coverage analysis, independent temporal/geographic validation, stakeholder review, and operational controls. No such validation is claimed here.

7. Repository and credential boundary

The public GitHub repository is a **Vite/React demonstration**. Its existing citizen, ASHA, and hospital views contain mock UI data. The Simulation Lab is the explicit ML experiment view. The Gemini chat is optional and currently runs from the browser if configured. A `VITE_GEMINI_API_KEY` value is exposed to users of the browser bundle, so it must never be treated as a private server credential, and the demo must not send real patient data. A key previously tracked in this repository was removed from current files, but its owner must rotate it because Git history retains it.

Generated run folders, Python environments, model files, raw historical source files, and old hybrid artifacts are kept out of Git. This prevents a teammate from mistaking an old model or unpublished local data for the checked-in synthetic experiment. The small checked-in configuration and source code are the reproducible dataset handoff.

8. Friend continuation checklist

1. Clone the repository. Run `npm ci`, `npm run build`, and `npm run lint`. From `ml/`, run `setup.ps1` and `run.ps1 -Mode Test` as shown above.
2. Run `run.ps1 -Mode Smoke`. Check `provenance_manifest.json`, positive counts, model artifacts, and the Simulation Lab's saved-model replay.
3. Review and freeze the full configuration and evaluation choices before opening the ten test worlds. Record the Git commit and configuration hash.
4. Run `run.ps1 -Mode Full` and preserve its timestamped run folder. Report measured world-level results, prevalence, misses, and false alerts even if they fall short of expectations.
5. Demonstrate the synthetic page locally and explain M1 onset lead separately from M2 reporting lead.
6. Finish the course report and presentation with the actual full-run data, the handwritten logistic derivation, algorithm comparisons, GitHub link, and limitations. Keep the final report within the course page limit.